9 — Convolutional networks 2: special blocks

Machine Learning (MPA-MLR)

Roman Jakubíček · Jiří Chmelík
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

Outline

1 · Skip connections

Short and long connections across layers, and the two ways of joining the data.

2 · Residual block

Identity plus a learned residual; why depth stops hurting so much.

3 · 1 × 1 convolution, bottleneck

Mixing features in depth; the cheap bottleneck block of ResNet-50.

4 · Dilated convolution

A larger receptive field with the same number of weights.

5 · Shrink and grow back

Autoencoders, unpooling, transposed convolution — the way to U-Net.

6 · Attention module

Where the network should look — channel and spatial attention.

1 · Skip connections

> Last time ReLU, batch normalization and initialization kept the gradient alive — and yet deeper plain networks still train worse.



Illustration generated by AI (FLUX.2)

Skip connections

Short and long

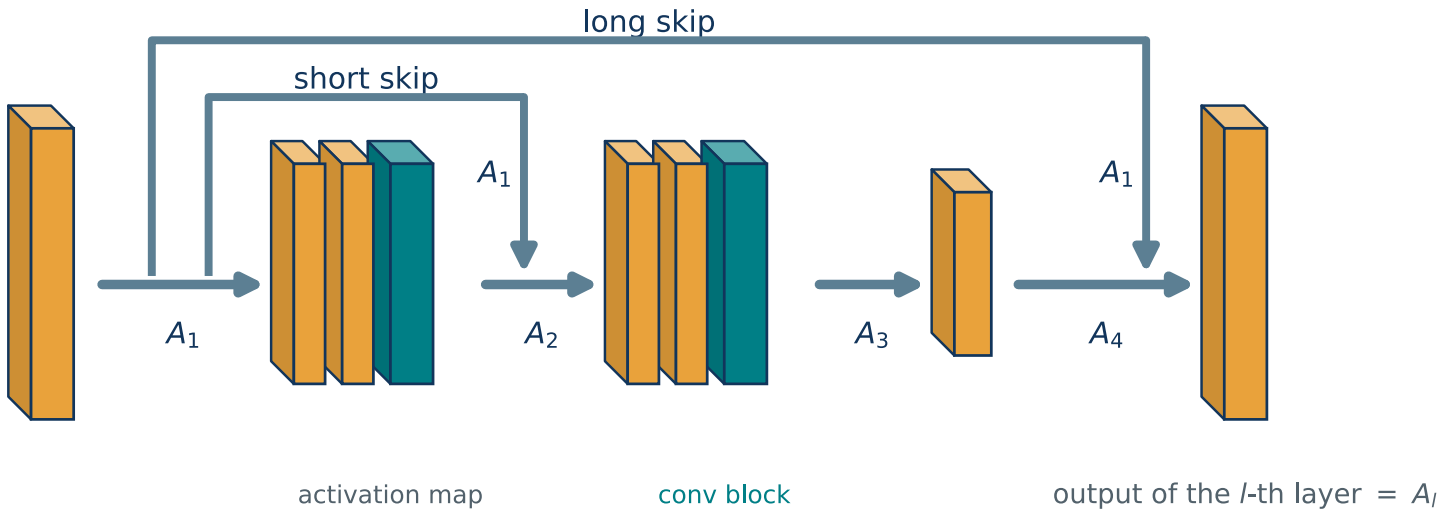
Short skips jump over a block, **long** skips run across the network

Information flow

Detailed early maps reach the deep layers

Gradient flow

Helps against **vanishing** gradients in deep nets



… but how can we join the data?

Why skip connections help

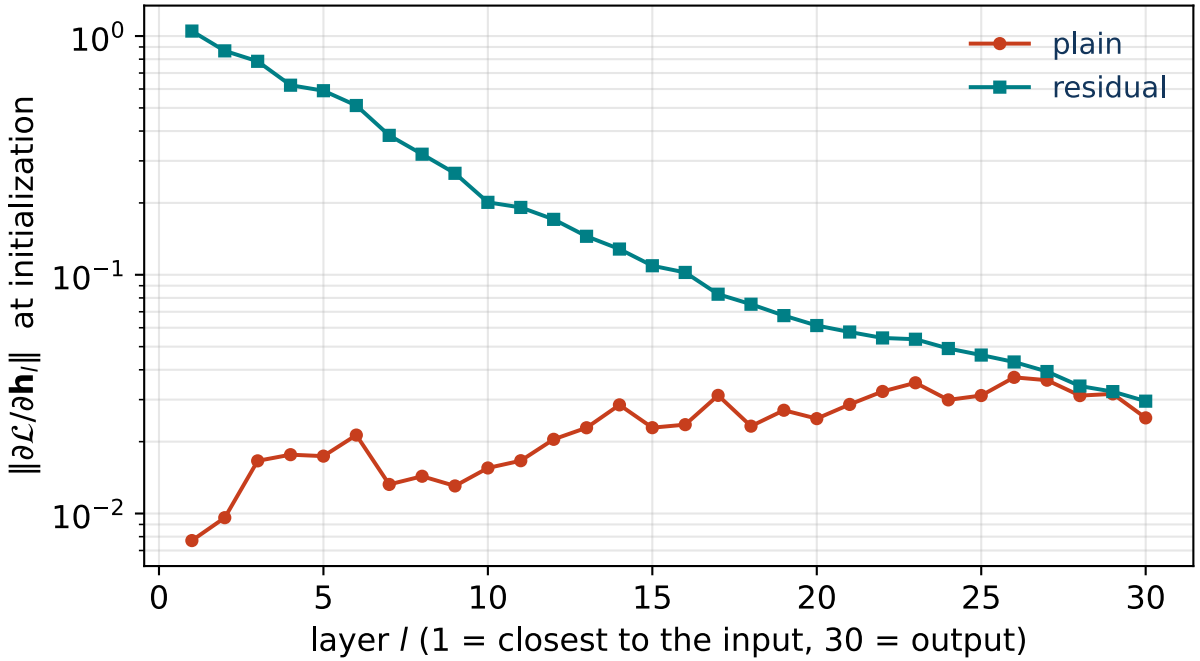
The gradient at layer l is a **product of Jacobians** of all the layers above — with every layer contracting a little, it shrinks exponentially.

A residual layer $\mathbf{h}_l = \mathbf{h}_{l-1} + f(W_l \mathbf{h}_{l-1} + \mathbf{b}_l)$ has

$$\frac{\partial \mathbf{h}_l}{\partial \mathbf{h}_{l-1}} = I + \operatorname{diag}\left(f'(\mathbf{z}_l)\right) W_l$$

— the identity adds a **direct term** I.

30 tanh layers at initialization: at the output 0.025 (plain) and 0.030 (residual); back at layer 1 the plain gradient is **3.3× smaller**, the residual one **35.5× larger**.

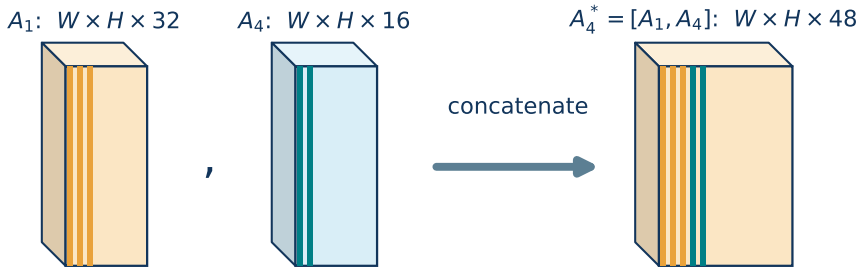


code example — residual block and the gradient

Joining data — concatenation

- **Concatenation:** stack the maps along the **depth**
- $W \times H$ must match; the depths **add up**

$$A_l^* = [A_i, A_l], \quad i < l, \qquad \text{e.g.} \quad A_4^* = [A_1, A_4]$$



✓ Nothing is lost

The next layer decides with its own weights how much of each source it uses.

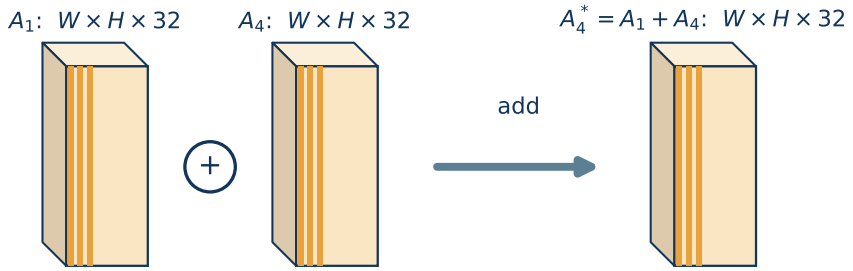
× The price

The output is deeper — more memory, more weights in the next convolution.

Joining data — addition

- **Addition:** element-wise sum — the identity bypasses the layers in between
- **all three** sizes must match; the output keeps the depth

$$A_l^* = A_i + A_l, \quad i < l, \qquad \text{e.g.} \quad A_4^* = A_1 + A_4$$



all three tensors have exactly the same shape

✓ No extra memory
The output keeps the shape — blocks can be chained.

× One shared space
Both tensors share one space; all three sizes must match.

Rule of thumb
U-Net concatenates, ResNet adds.

2 · Residual block

> The block that made 100-layer networks trainable — and it is one addition.



Illustration generated by AI (FLUX.2)

Residual block

Input
map $\mathbf{x}$

Output
$\mathcal{F}(\mathbf{x}) + \mathbf{x}$, the **same shape**

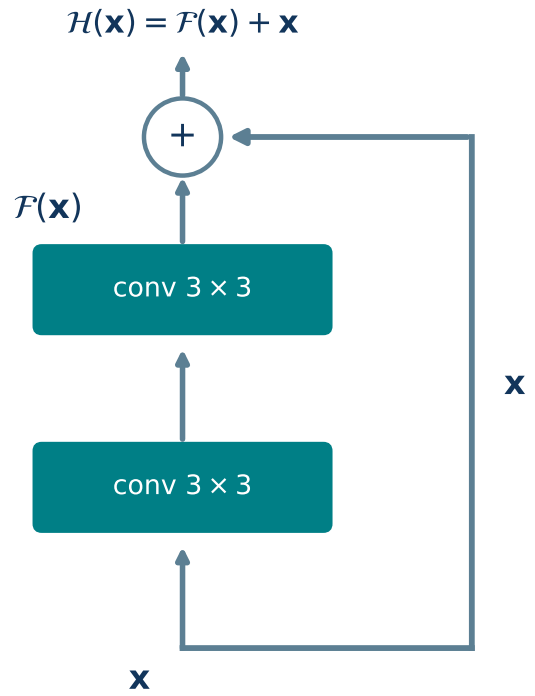
Hyperparameters
layers in the branch; the variant

Learned parameters
the branch only; the identity **none**

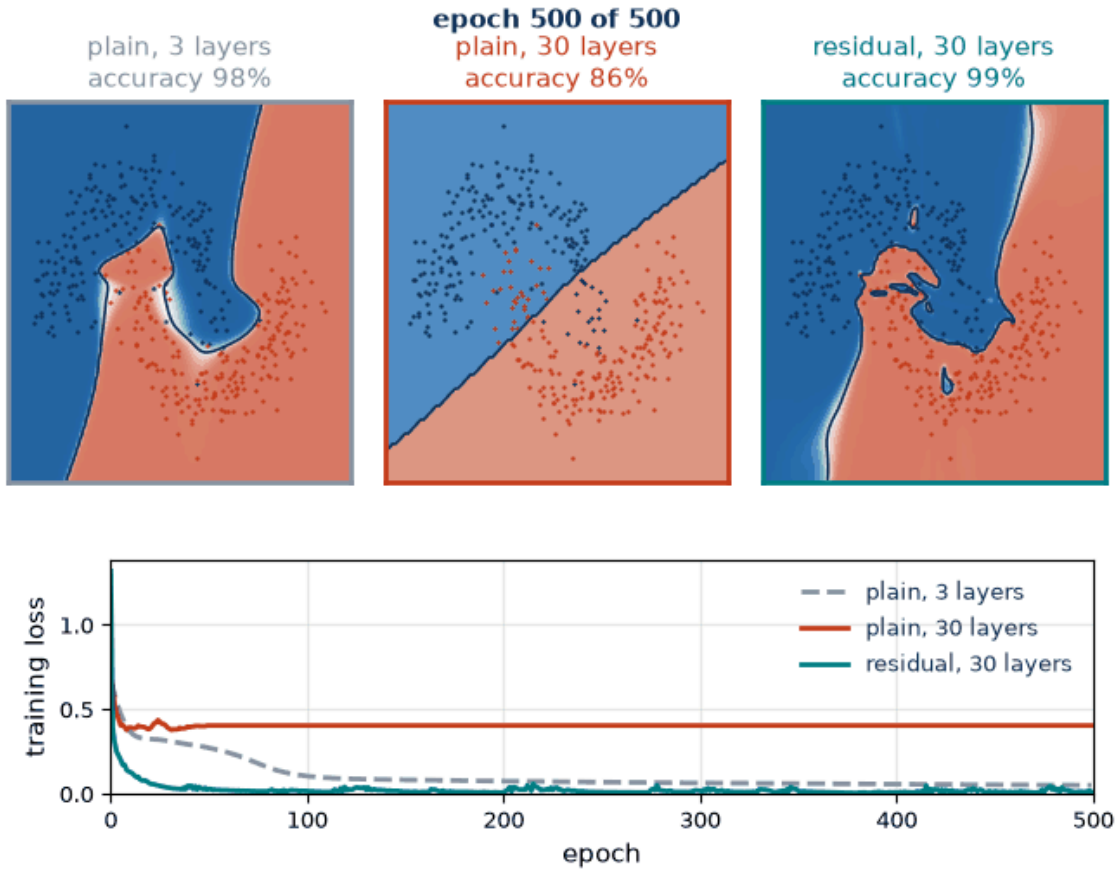
- **Deeper ≥ shallower** in principle — the extra layers could be the identity
- …but deeper plain nets are **harder to optimize**
- The convolutions learn the **residual**, $\mathcal{H}(\mathbf{x})$ is the desired mapping:

$$\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$$

Identity = all branch weights zero — easy to reach; the gradient flows through $\mathbf{x}$.



Deeper is not automatically better



Three `tanh` networks — the same data, optimizer and initial weights (where they share layers):

3 layers, plain

loss 0.049, accuracy $97.5\,\%$

30 layers, plain

stuck at 0.40 and $86\,\%$: *deeper is worse*

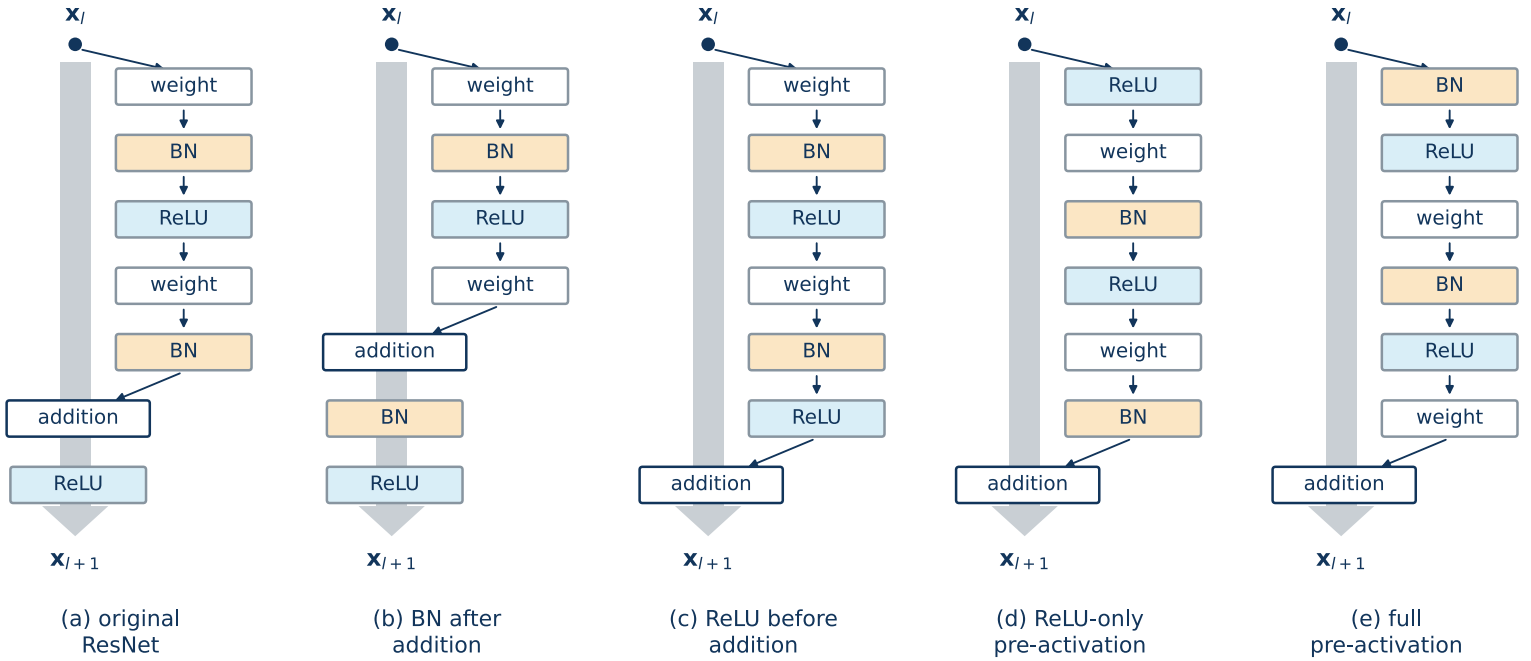
30 layers, residual

`h = h + tanh(W @ h + b)`: loss 0.018, accuracy $99.25\,\%$

code example — residual block and the gradient

Residual block — variations

- (a), (b): a ReLU or batch normalization (BN) **after the addition**
- (c)–(e): a **clean identity**; they differ in the order **inside the branch**
- a **1 × 1 convolution** on the identity branch when the depths differ



Drawn after He et al., *Identity Mappings in Deep Residual Networks*, ECCV 2016 (arXiv:1603.05027), Fig. 4.

3 · 1 × 1 convolution and bottleneck

> A convolution that sees a single pixel — and is one of the most useful blocks there is.

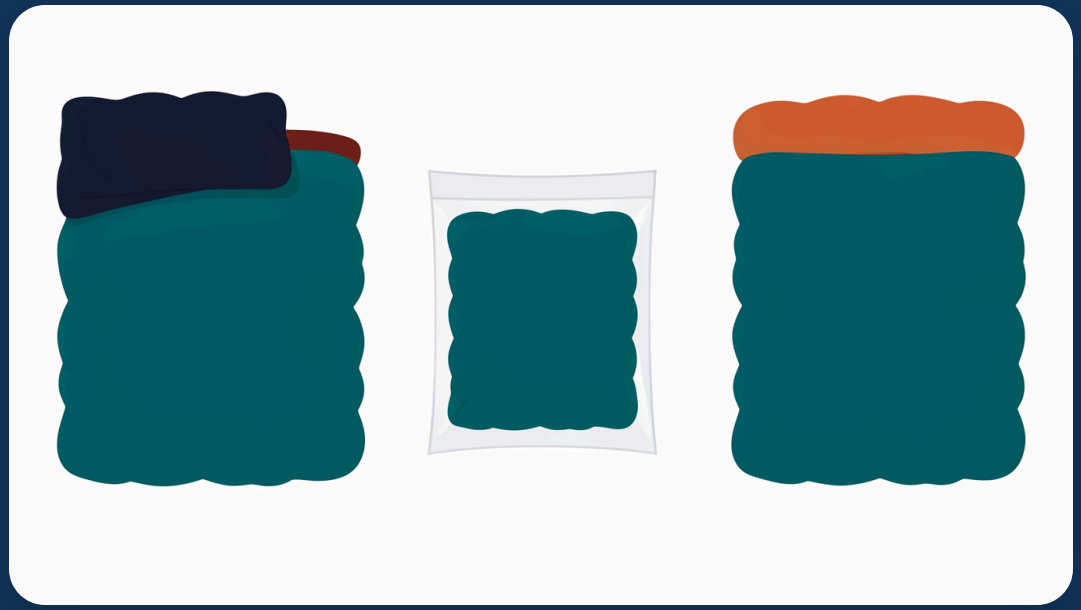


Illustration generated by AI (FLUX.2)

1 × 1 convolution layer

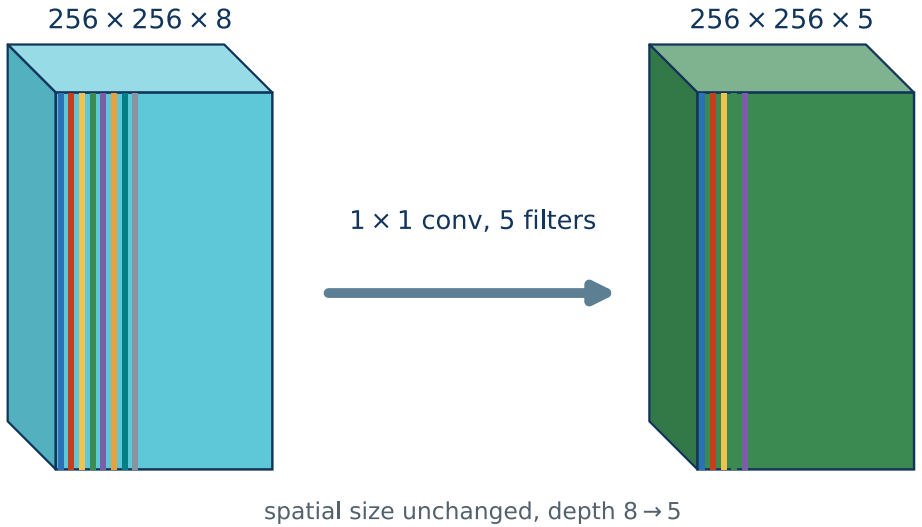
Input
map $W \times H \times D$

Output
$W \times H \times K$ — same $W \times H$, new depth

Hyperparameters
the number of filters K

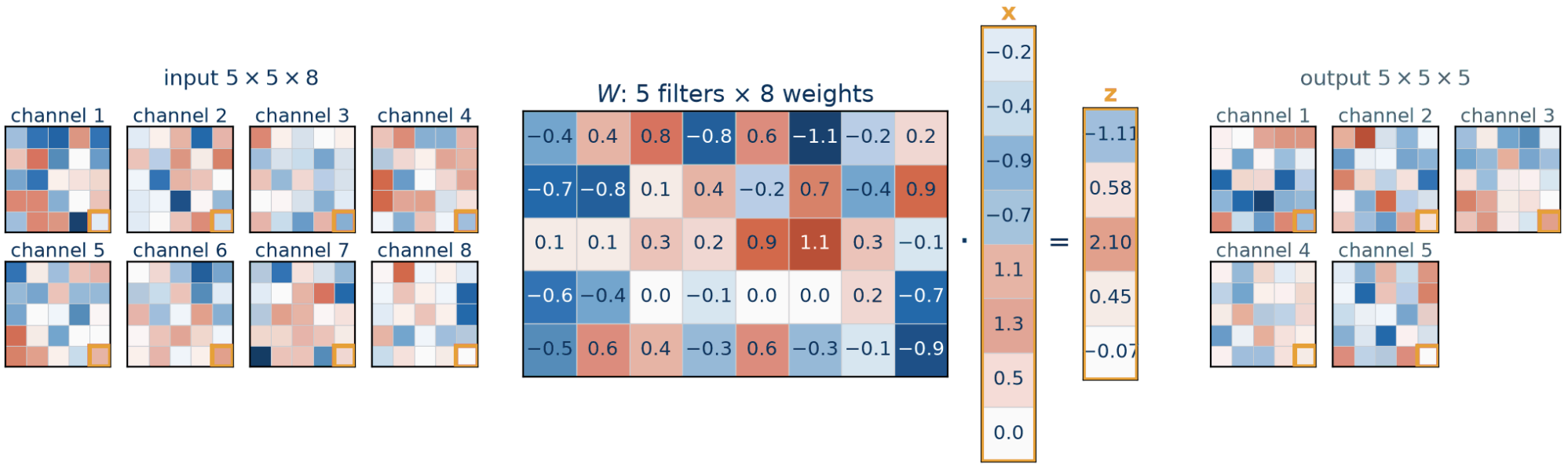
Learned parameters
$K \cdot D$ weights $+ K$ biases

- also called **pointwise** convolution
- mixes features **in depth**, not in space
- each output map = a **linear combination** of the input maps



spatial size unchanged, depth 8 → 5

1 × 1 convolution — pixel by pixel



pixel (5, 5): its 8 features **x** times the same 5 × 8 matrix give its 5 new features $\mathbf{z} = W\mathbf{x}$ (bias 0 here) — no neighbour is looked at

The same layer on a 5×5 crop: at every pixel $\mathbf{z} = W\mathbf{x} + \mathbf{b}$ with **one** 5×8 matrix — the 40 weights of the table on the next slide.

Why use a 1 × 1 convolution?

Uses

- **new combinations** of features
- **reduce** or **expand** the number of features
- **speed-up** by reducing the depth
- **depthwise separable** convolution (MobileNet)
- a **fully connected** layer, written as a convolution

Cost

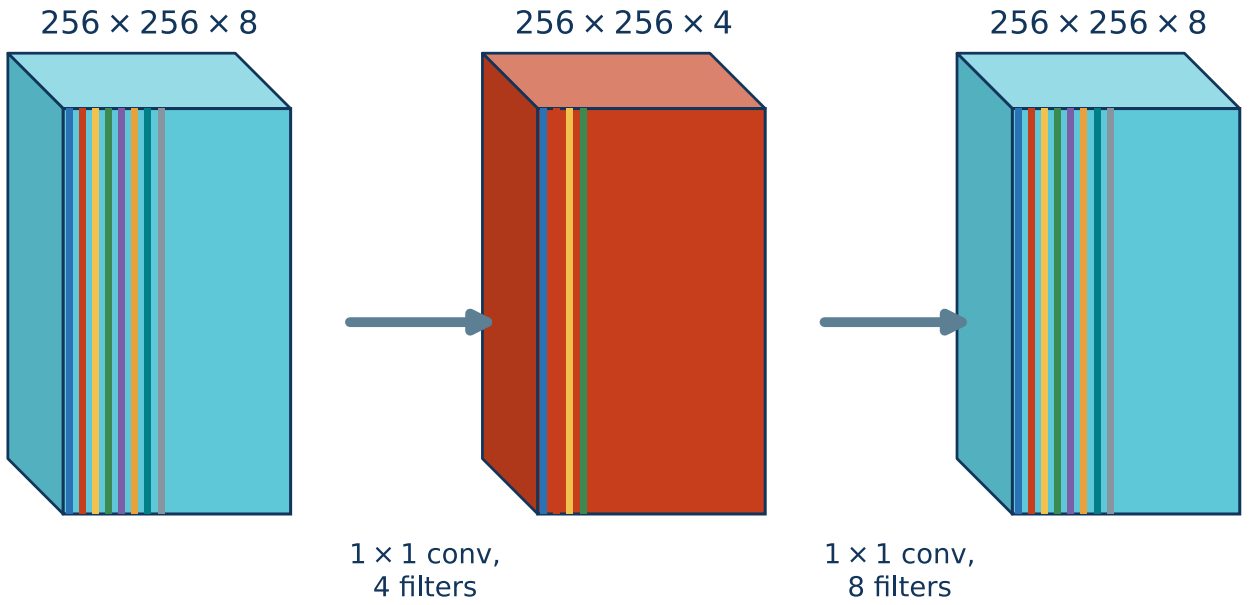
For the layer from the two previous slides:
$256 \times 256 \times 8 \to 256 \times 256 \times 5$

kernel	weights	biases	MACs / pixel
1×1	$1 \cdot 1 \cdot 8 \cdot 5 = 40$	5	40
3×3	$3 \cdot 3 \cdot 8 \cdot 5 = 360$	5	360

MAC = multiply–accumulate. Nine times fewer weights and operations — but no spatial context.

A 1×1 convolution applied to a $1 \times 1 \times D$ tensor **is** a fully connected layer: the same weights, the same output.

Bottleneck — reduction of the feature dimension



524 288 → 262 144 values (2× fewer)

The spatial size stays, the depth shrinks — done by the 1×1 convolution: $524\,288 \to 256 \cdot 256 \cdot 4 = 262\,144$ values, only **2×** fewer. The saving is in the *next* layer, which works on 4 features instead of 8.

Bottleneck layer

- **1 × 1** narrows the depth, the **3 × 3** works on it, **1 × 1** widens it back
- a large **saving** in weights and computation
- the bottleneck block of **ResNet-50** and deeper; a 1×1 reduction also in GoogLeNet/Inception and DenseNet

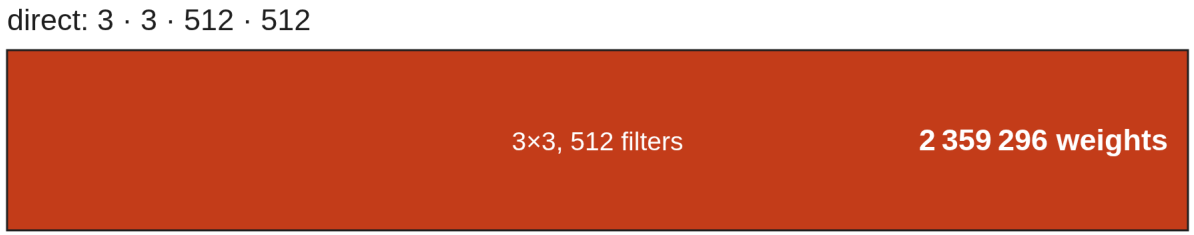


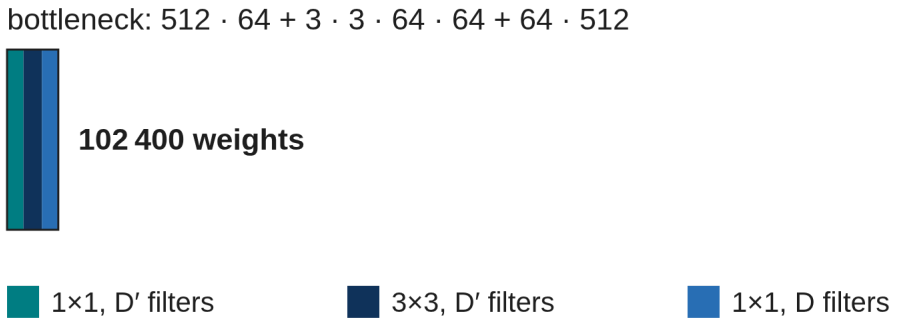
Bottleneck layer — what it saves

	weights	MACs for a 300×300 map
3×3 conv, 512 filters (direct)	$3 \cdot 3 \cdot 512 \cdot 512 = 2\,359\,296$	$2.12 \cdot 10^{11}$
1×1 conv, 64 filters	$512 \cdot 64 = 32\,768$	$2.95 \cdot 10^{9}$
3×3 conv, 64 filters	$3 \cdot 3 \cdot 64 \cdot 64 = 36\,864$	$3.32 \cdot 10^{9}$
1×1 conv, 512 filters	$64 \cdot 512 = 32\,768$	$2.95 \cdot 10^{9}$
bottleneck, total	$102\,400$	$9.22 \cdot 10^{9}$

23 × fewer weights and 23 × less arithmetic — for the same input and output shape.

Playground: what does the bottleneck save?





Direct 3 × 3 convolution **D → D** against the bottleneck **D → D′ → D′ → D** (1 × 1, 3 × 3, 1 × 1). Biases left out, as in the table.

depth of the map **D** = 512

bottleneck width **D′** = 64

map size **W** = **H** = 300

512 → 64 (slide) | 256 → 64 (ResNet-50) | D′ = D

direct: **2 359 296** weights, $2.12 \cdot 10^{11}$ MACs

bottleneck: **102 400** weights, $9.22 \cdot 10^{9}$ MACs

saving: **23.04 ×** fewer weights and MACs

MACs = weights × W · H output pixels; the ratio does not depend on the map size.

4 · Dilated convolution

> Nine weights, but a window as big as you like — and the output keeps its full resolution.



Illustration generated by AI (FLUX.2)

Dilated convolution

Input
map $W \times H \times D$

Output
the same $W \times H$ — **no down-sampling**

Hyperparameters
F, dilation δ, K;
$P = (F-1)\delta/2$

Learned parameters
as an ordinary convolution — δ adds **none**

Aggregates **context at several scales** — a larger receptive field without down- and up-sampling.

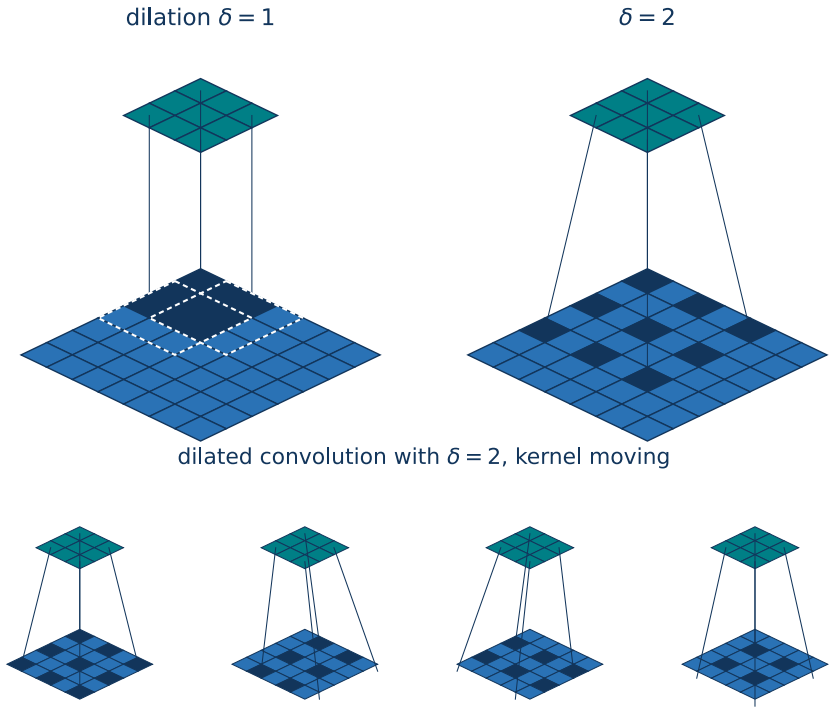
Standard convolution

$$z(i,j) = \sum_u \sum_v x(i+u,\ j+v)\ w(u,v)$$

Dilated convolution, dilation δ

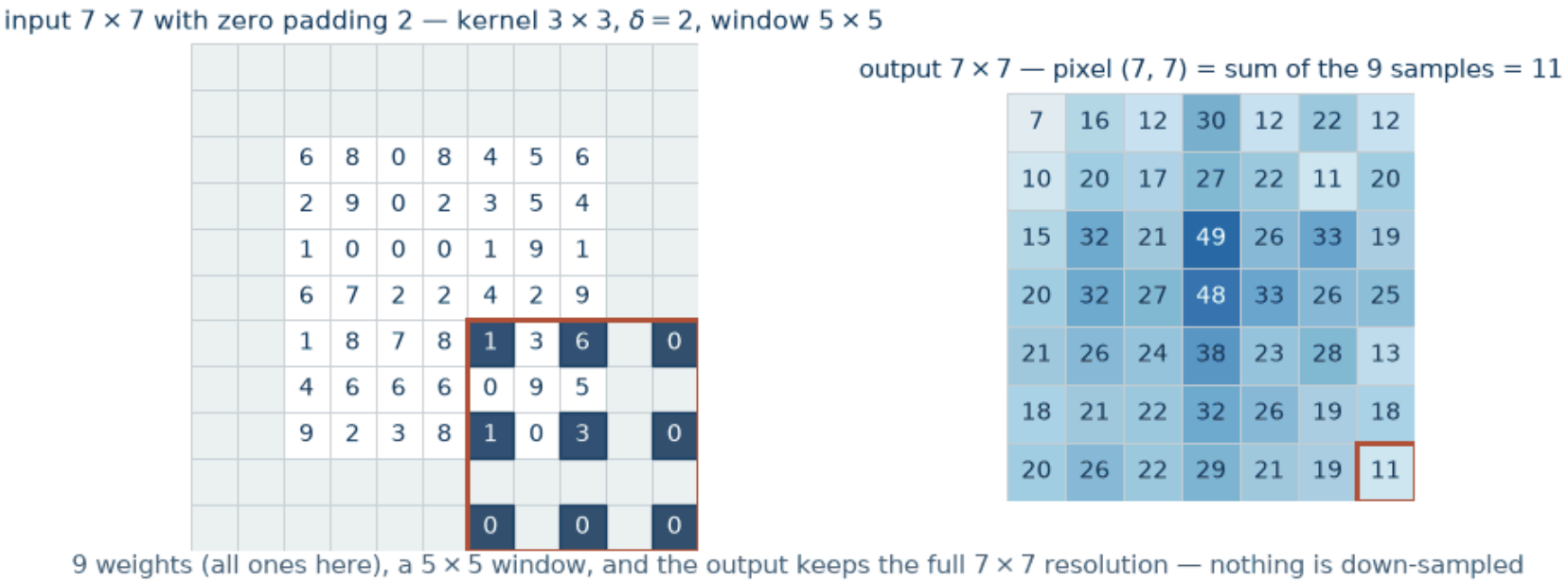
$$z(i,j) = \sum_u \sum_v x(i+\delta u,\ j+\delta v)\ w(u,v)$$

with $u, v = -1, 0, 1$ for a 3×3 kernel.



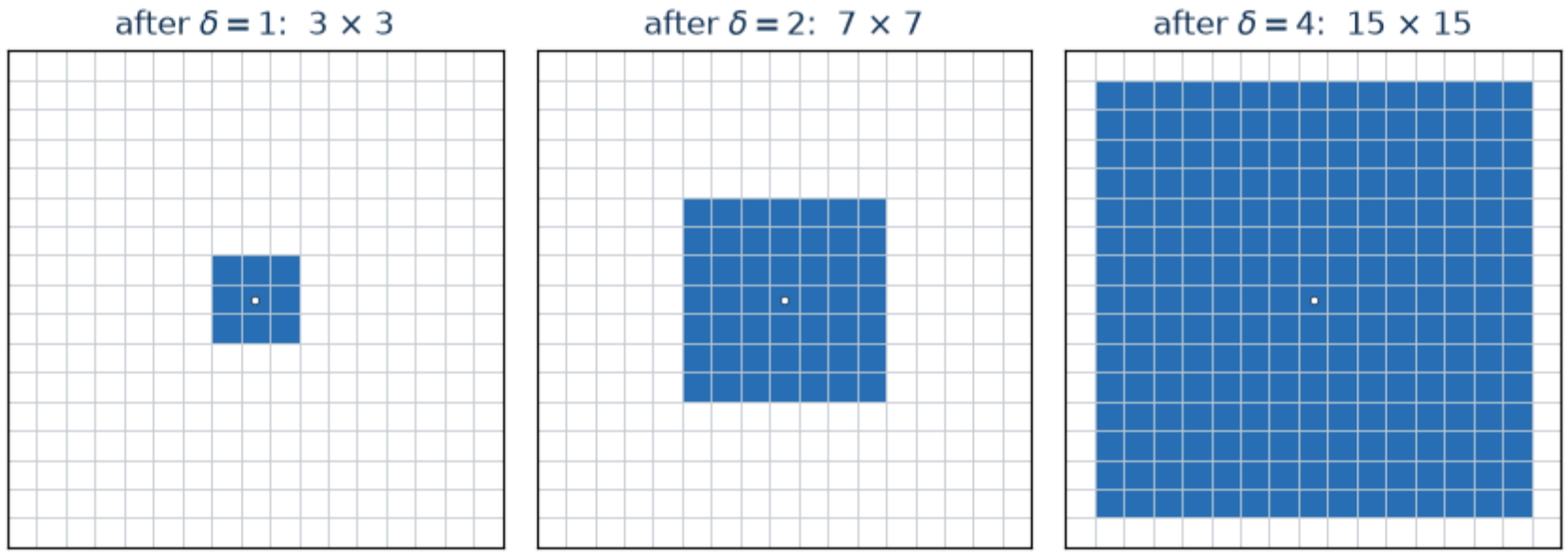
Drawn after Yu & Koltun, *Multi-Scale Context Aggregation by Dilated Convolutions*, ICLR 2016 (arXiv:1511.07122).

Dilated convolution — the kernel sliding



All nine weights are 1, so every output pixel is the sum of the nine sampled values — check any of them by hand. Window 5×5, zero padding $P = 2$, and the output keeps the **same 7×7 size**.

Dilated convolution — the receptive field



each layer copies the previous field to its 9 taps (red × = the tap, orange = the copy just added)

Dilated, δ = 1, 2, 4

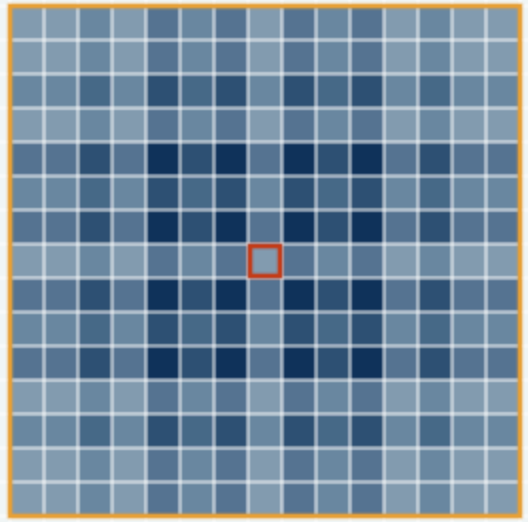
A 3×3 kernel with dilation δ covers $(F-1)\delta + 1$ pixels; three of them reach $3 \to 7 \to 15$ — with $3 \times 9 = 27$ weights.

Ordinary 3 × 3

Three of them reach only $3 \to 5 \to 7$; a 15×15 kernel would need 225 weights.

code example — dilated convolution and the receptive field

Playground: choose the dilations



Three 3 × 3 convolutions in a chain. The map shows the input pixels that reach the **centre** output pixel; darker = more paths. Pink = a hole inside the field.

layer 1: $\delta_1 = 1$

layer 2: $\delta_2 = 2$

layer 3: $\delta_3 = 4$

1, 1, 1 | 1, 2, 4 | 2, 2, 2 ☐ path counts

receptive field layer by layer: 3 → 7 → **15 × 15**
pixels the centre really sees: **225** of 225 — no holes
weights: 3 × 9 = **27** (a single kernel that wide: 225)
most paths into one input pixel: 9

5 · Shrink and grow back

> Squeeze the input through a narrow spot, then grow it back — segmentation and generation need the way back up.



Illustration generated by AI (FLUX.2)

Bottleneck — autoencoders

- a **narrow layer** limits what can pass → a learned **compression**
- a **trade-off** between accuracy and compression
- trained **without labels** — the input is the target:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \sum_{i=1}^{n} \|\mathbf{x}_i - \hat{\mathbf{x}}_i(\mathbf{w})\|_2^2$$

- without the bottleneck the network could just **copy** the input

Applications: dimensionality reduction · the encoder–decoder shape (segmentation) · bottleneck layers (the third block).

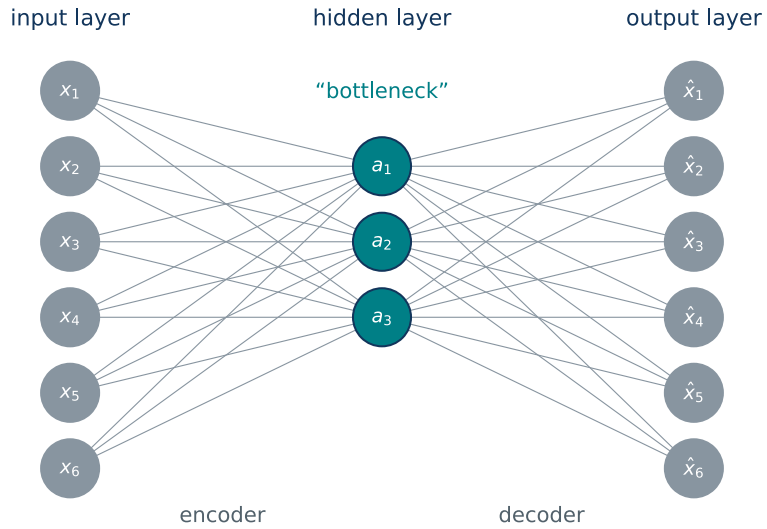
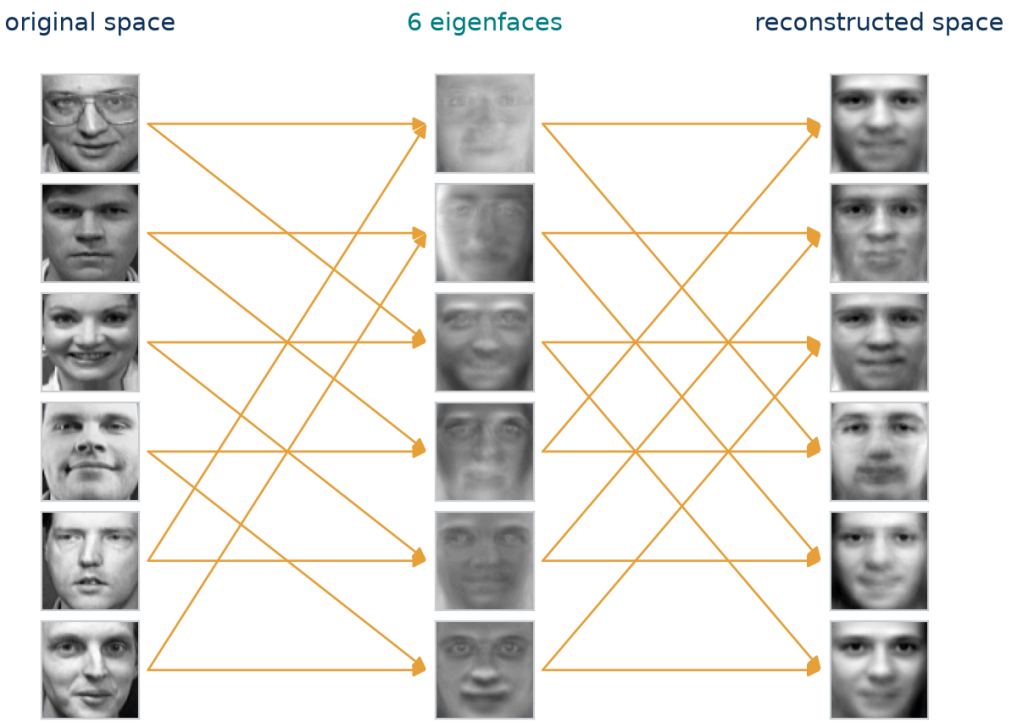




Illustration generated by AI (FLUX.2)

Bottleneck — feature reduction by PCA

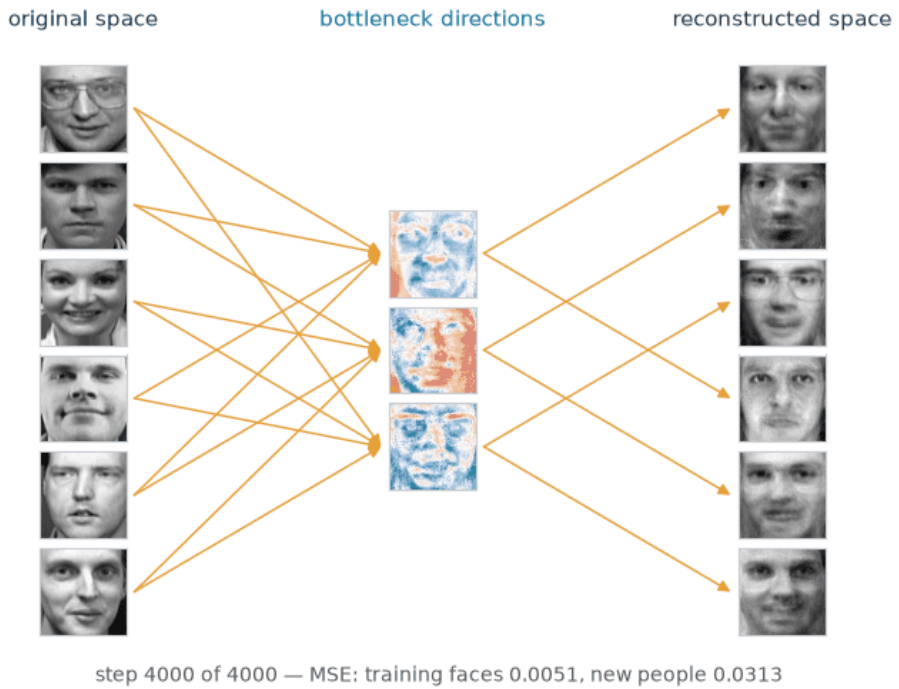


PCA-6 fitted on 30 people, shown on 6 new people — MSE on the 10 new people 0.0107

Principal component analysis (PCA) fitted on **30 people** (300 faces), shown on **10 new people**.

Olivetti faces (AT&T Laboratories Cambridge), computed by the code example.

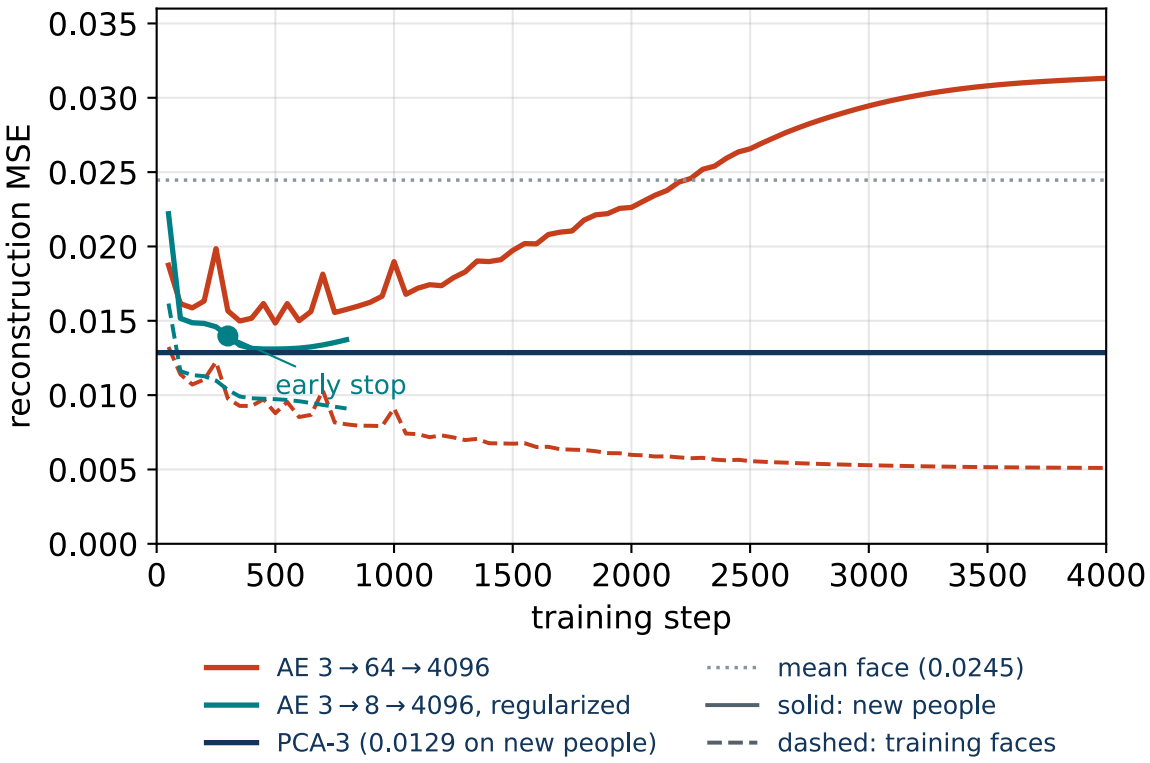
Bottleneck — feature reduction by a neural net



A linear encoder, a nonlinear decoder ($3 \to 64 \to 4096$, tanh). Training faces: mean squared error (MSE) 0.0051 (PCA-3: 0.0100). **New people:** 0.0313 — worse than the mean face (0.0245).

Olivetti faces (AT&T Laboratories Cambridge), computed by the code example.

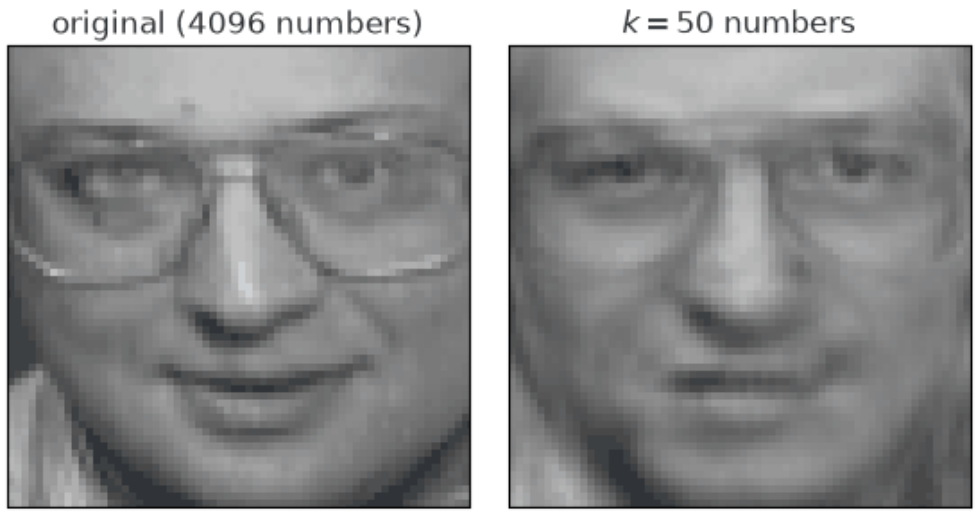
Bottleneck — and on new faces?

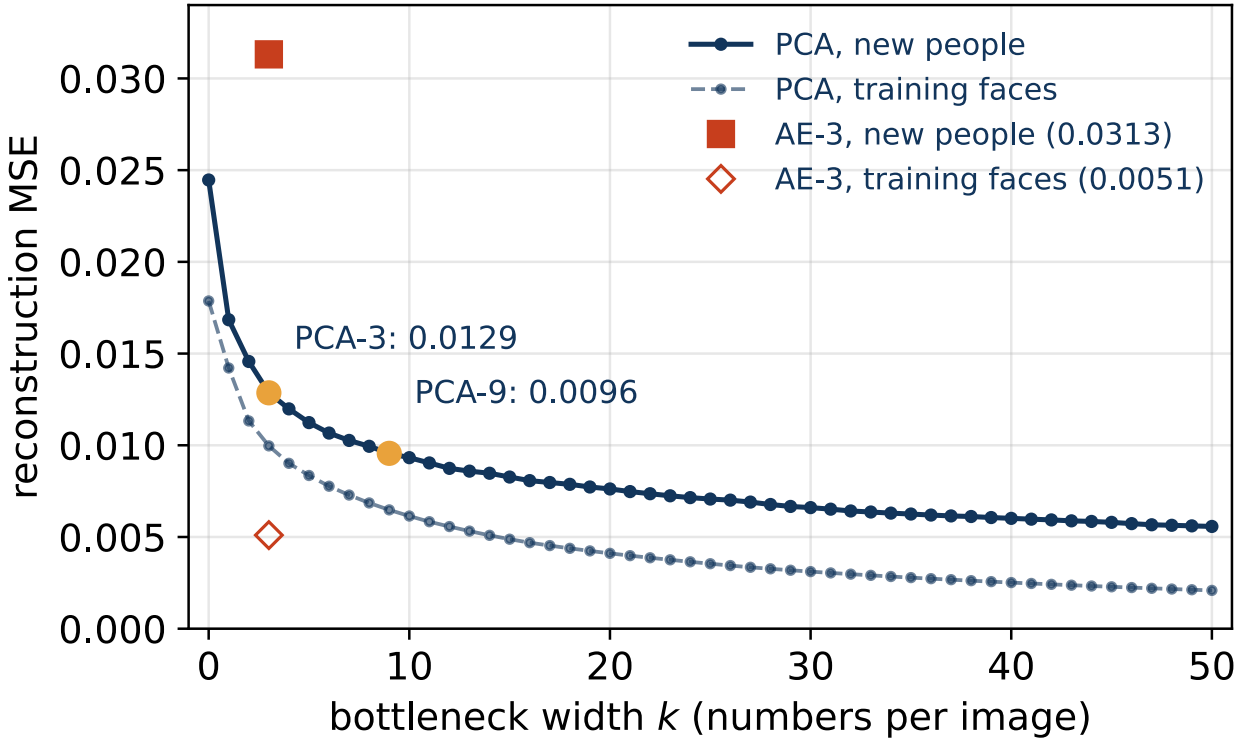


MSE	training	new people
mean face	0.0179	0.0245
PCA-3	0.0100	**0.0129**
PCA-9	0.0065	0.0096
autoencoder (AE) $3 \to 64 \to 4096$	**0.0051**	0.0313
AE $3 \to 8 \to 4096$, regularized	0.0104	0.0140

A bottleneck alone does **not** prevent overfitting. Regularized, the autoencoder generalizes — but with 400 faces, linear PCA is hard to beat.

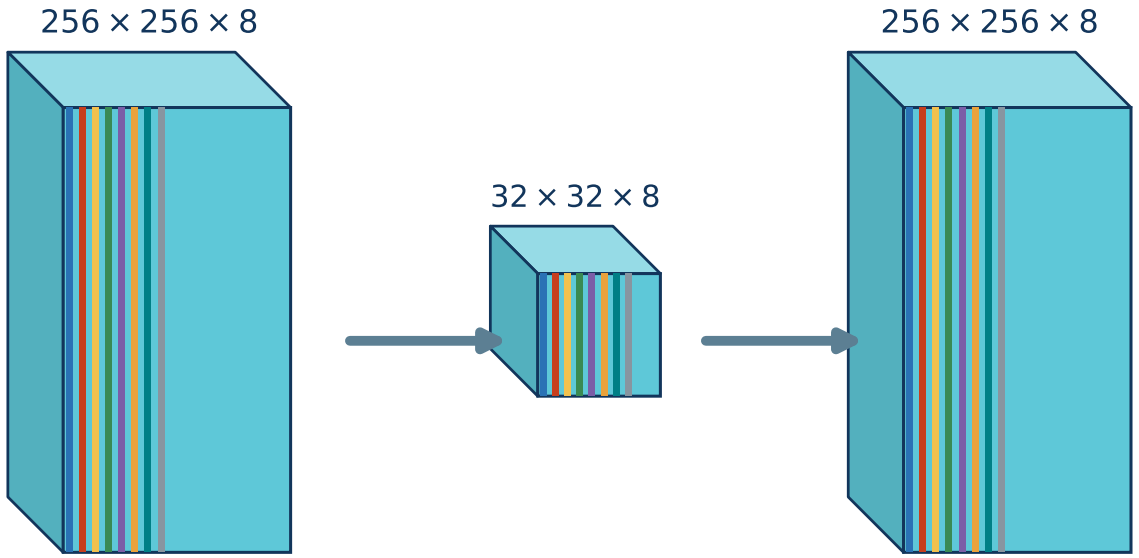
Bottleneck — how narrow can it be?





$k = 0$ is the mean face; on the training faces each extra component removes a little less error; on new people the curve lies higher and flattens out too.

Bottleneck — reduction of the spatial dimension



524 288 → 8 192 values (64× fewer)

Down-sampling (pooling or a strided convolution) shrinks W and H and keeps the depth: $256 \cdot 256 \cdot 8 = 524\,288$ values become $32 \cdot 32 \cdot 8 = 8\,192$ — **64× fewer**. Doubling the depth of the narrow map to 16 still gives a 32× compression.

Up-sampling layers

Segmentation and **generation** need maps back at full resolution.

Unpooling

puts the maxima back to their **positions** (switches)

Transposed convolution

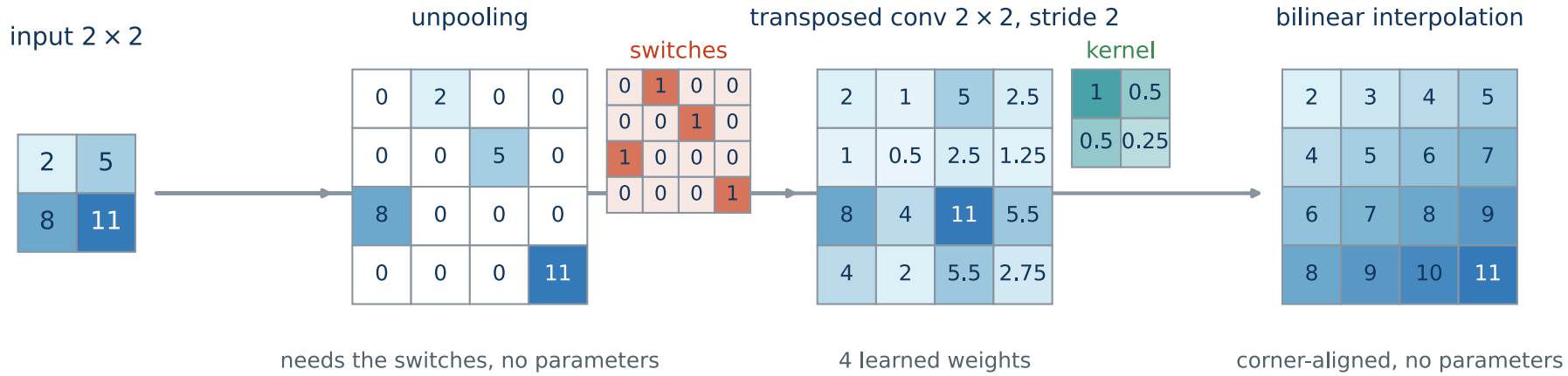
a convolution run backwards, **learned** kernel

Interpolation

bilinear or nearest, **nothing to learn**

Only the middle one has parameters. Unpooling needs the positions remembered from pooling; interpolation needs nothing at all.

Up-sampling — the same map three ways



The same 2×2 input, three different 4×4 outputs — only the transposed convolution has **learned** weights.

Unpooling layer (max)

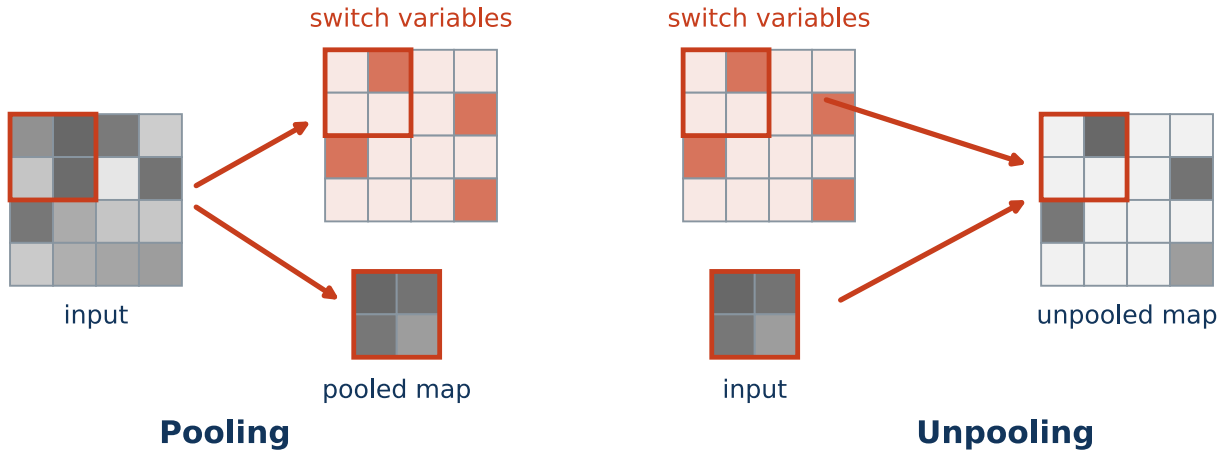
Input
pooled map + **switch variables** (where each maximum was)

Output
the size before pooling; the maximum at its position, **zeros** elsewhere

Hyperparameters
window and stride of the paired max pooling

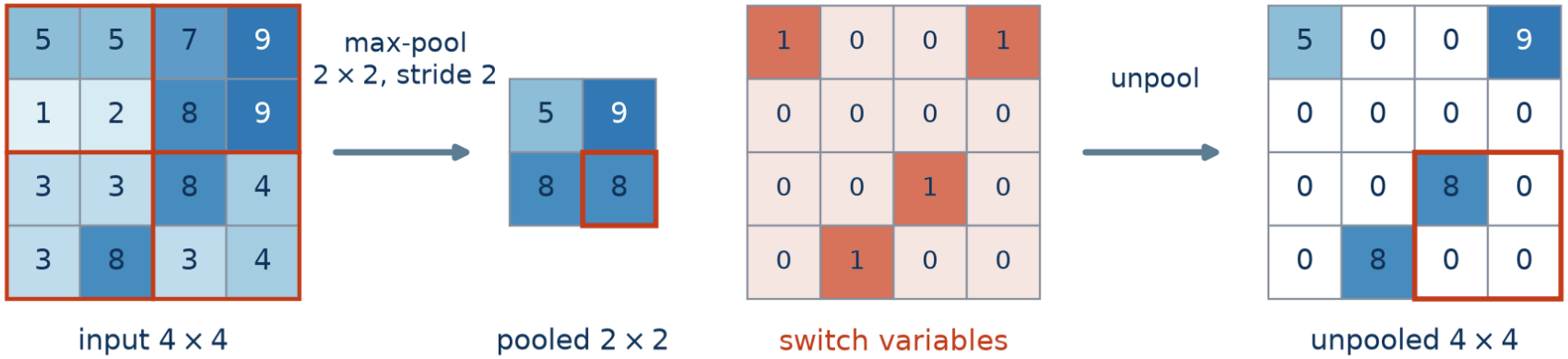
Learned parameters
none

- reverses **max pooling** — the **positions** only; after average pooling, interpolate instead



Drawn after Zeiler & Fergus, *Visualizing and Understanding Convolutional Networks*, ECCV 2014 (arXiv:1311.2901).

Unpooling — a worked example



Max pooling 2×2 with stride 2 keeps 4 of the 16 values, so unpooling can put back **4 numbers out of 16** — the rest is zero. The switch variables cost one index per pooled value.



Illustration generated by AI (FLUX.2)

Convolution as a matrix product

The convolution of the previous lecture (one channel):

$$z(i,j) = \sum_u \sum_v x(i+u,\ j+v)\ w(u,v)$$

It is **linear** in X → it is a **matrix product**:

$$\mathrm{vec}(Z) = C\ \mathrm{vec}(X)$$

X input, Z output, C the **convolution matrix**.

No inverse with stride S > 1
C has $\lceil M/S \rceil^2$ rows and M^2 columns — it is **not square**.



Illustration generated by AI (FLUX.2)

But we can use the convolution matrix to up-sample Z back to the size of X.

Transposed convolution — padding and vectorization

- **transposed convolution** undoes the *shape* change of a strided convolution — sometimes wrongly called "deconvolution"
- first write the convolution as a matrix product:
- **pad** $N \times N$ to $M \times M, M = N + (F - 1)$
- **vectorize** row by row → length M^2

Kernel W ($F = 3$):

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \\ w_{3,1} & w_{3,2} & w_{3,3} \end{bmatrix}$$

Padded image and its vectorization:

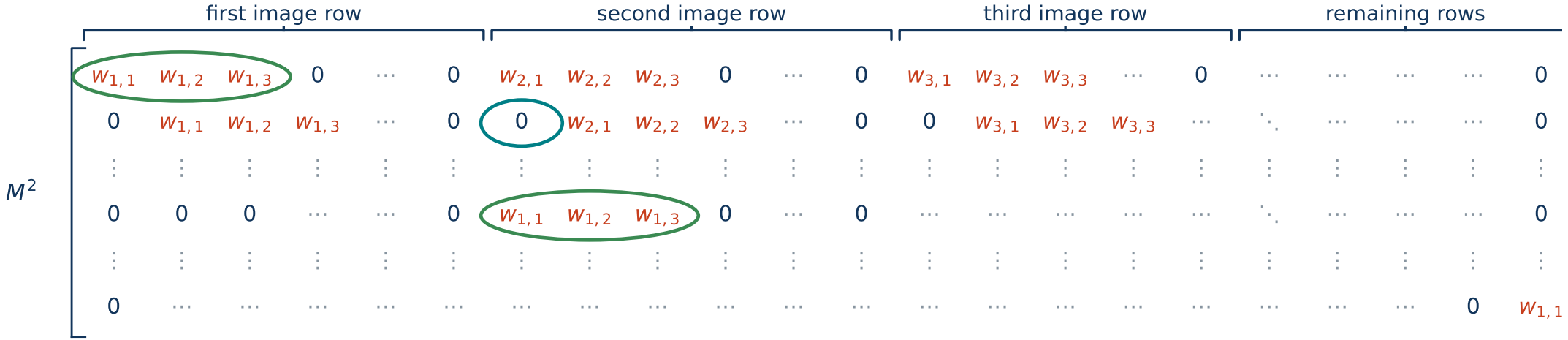
$$X = \begin{bmatrix} x_{1,1} & \cdots & x_{1,M} \\ \vdots & \ddots & \vdots \\ x_{M,1} & \cdots & x_{M,M} \end{bmatrix} \Rightarrow \operatorname{vec}(X) = \begin{bmatrix} x_{1,1} \\ x_{1,2} \\ \vdots \\ x_{M,M} \end{bmatrix} M^2$$

In the matrix derivation the window is anchored at its **top-left corner**: $w_{1,1}$ multiplies $x(i, j)$, i.e. the offsets u, v run from 0 to $F - 1$.

Convolution matrix C — stride 1 and stride 2

Each row of C is one position of the kernel; the three groups of weights are the three kernel rows, M columns apart.

stride 1

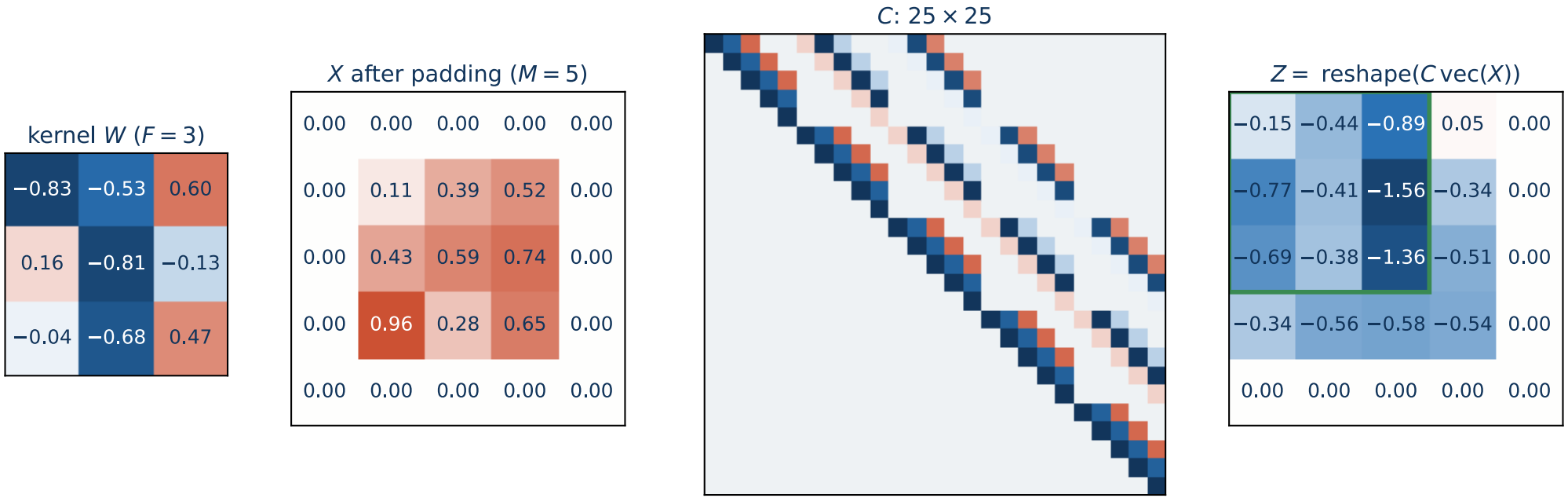


stride 2 (even M): every other kernel position — the rows of the skipped positions are removed



M^2 columns = the pixels of vec(X)

Convolution matrix — a worked example

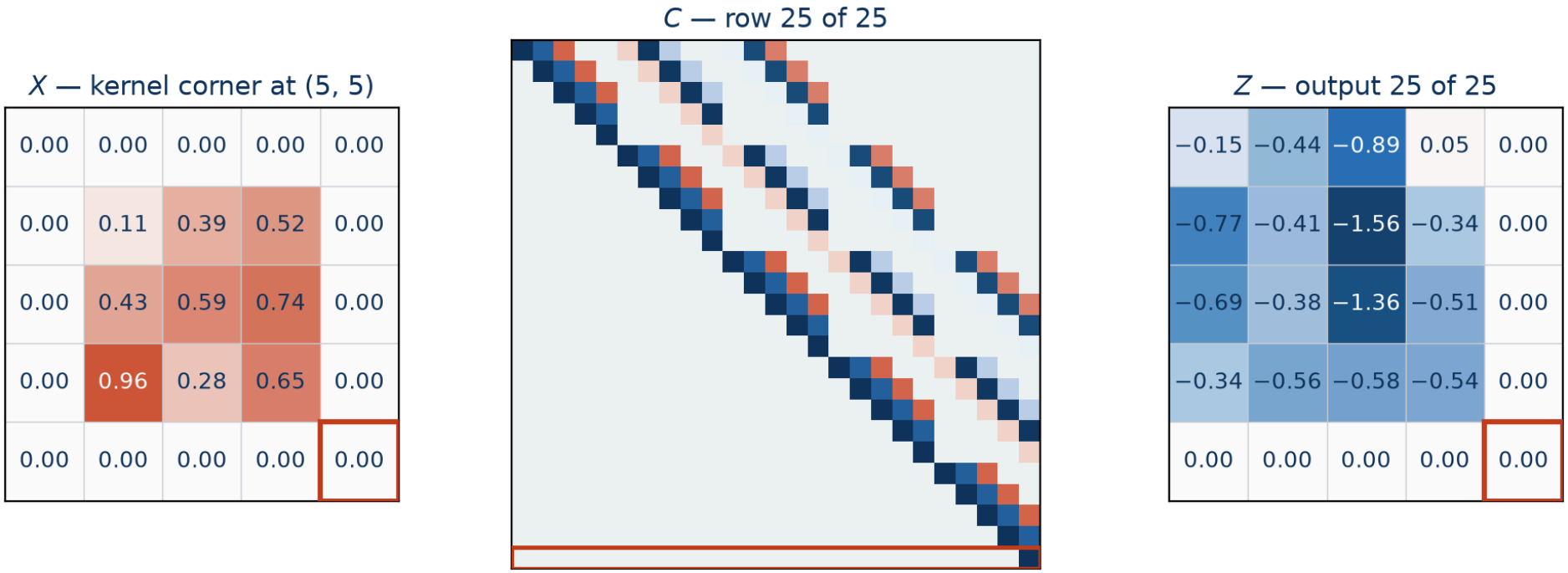


green block of Z = the same-size output of the unpadded 3 × 3 image; checked against scipy.signal.correlate2d

$N = 3, F = 3$, so $M = 5$ and C is 25×25 with at most 9 non-zeros per row — 144 of the 625 entries, $23\,\%$. Here C is square and, because $w_{1,1} \neq 0$, invertible: $\det C = w_{1,1}^{25}$, but $\operatorname{cond} C = 147$.

code example — the convolution matrix, checked against `scipy.signal.correlate2d`

Convolution matrix — built row by row



one row of C = one kernel position = one output pixel; for the last rows the kernel hangs over the edge

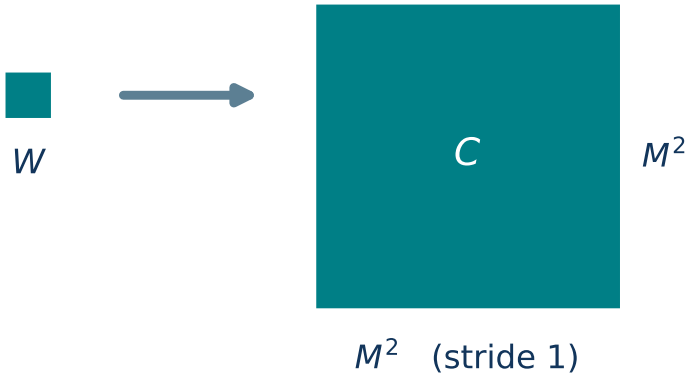
The kernel with its corner at (i, j) is row $(i-1)M + j$ of C; its nine weights land in the columns of the pixels it covers. Positions that hang over the edge have fewer entries — the last one only $w_{1,1}$.

Transposed convolution — the two steps

1. vectorize the $M \times M$ map X row by row



2. build the sparse convolution matrix C from the kernel W



convolution matrix for stride 2



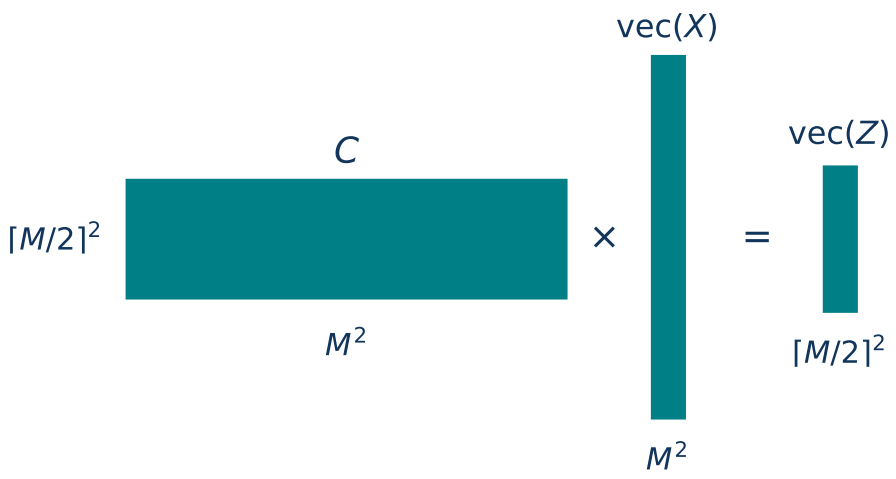
fewer rows = fewer output positions

For a real 256×256 map, C would have $4 \cdot 10^9$ entries — the matrix form is a derivation, not an implementation.

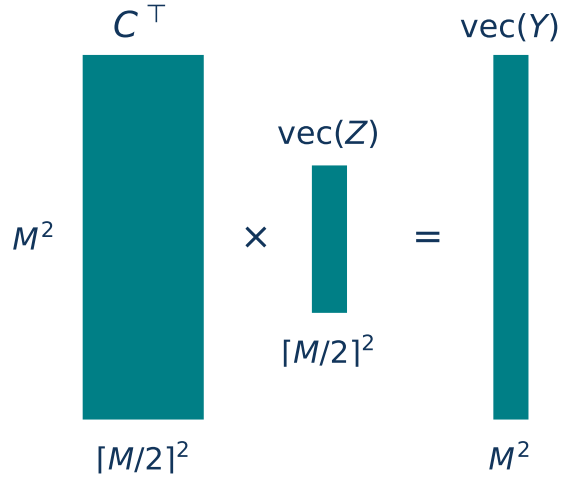
Transposed convolution — up-sampling

Up-sampling: multiply by $C^\top$ — the length goes back to M^2. In a network C is built from a **learned** kernel: the same sparse shape, only F^2 free weights.

convolution, stride 2



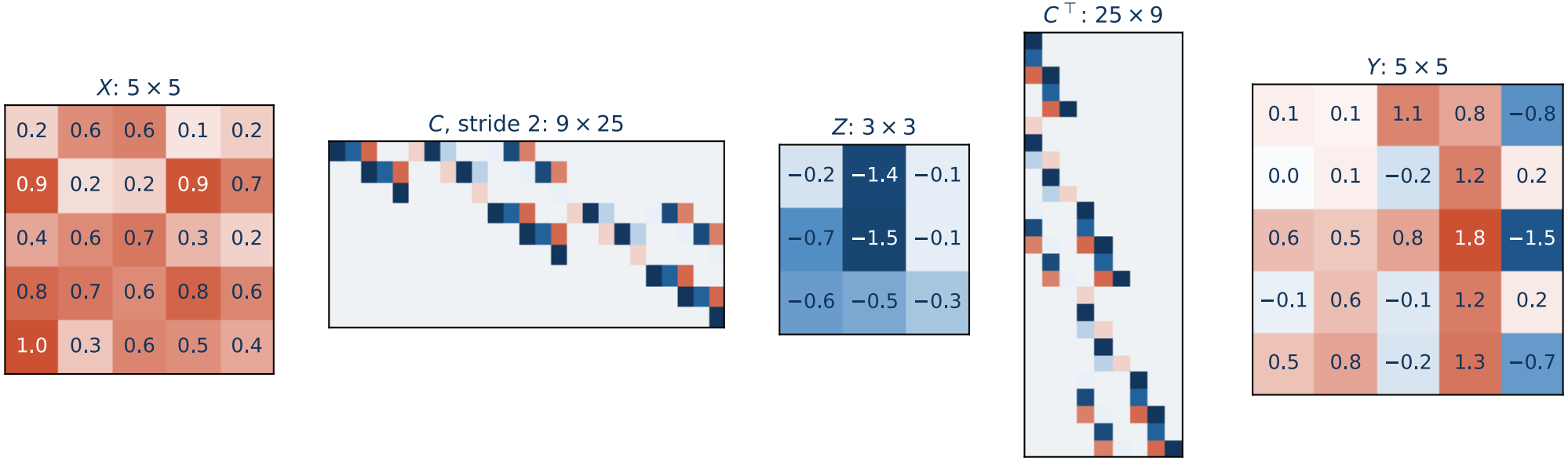
transposed convolution, stride 2



C of the same sparse shape, built from a learned kernel

convolution: $\mathrm{vec}(Z) = C\ \mathrm{vec}(X)$ transposed: $\mathrm{vec}(Y) = C^\top \mathrm{vec}(Z)$

Transposed convolution — the example continued

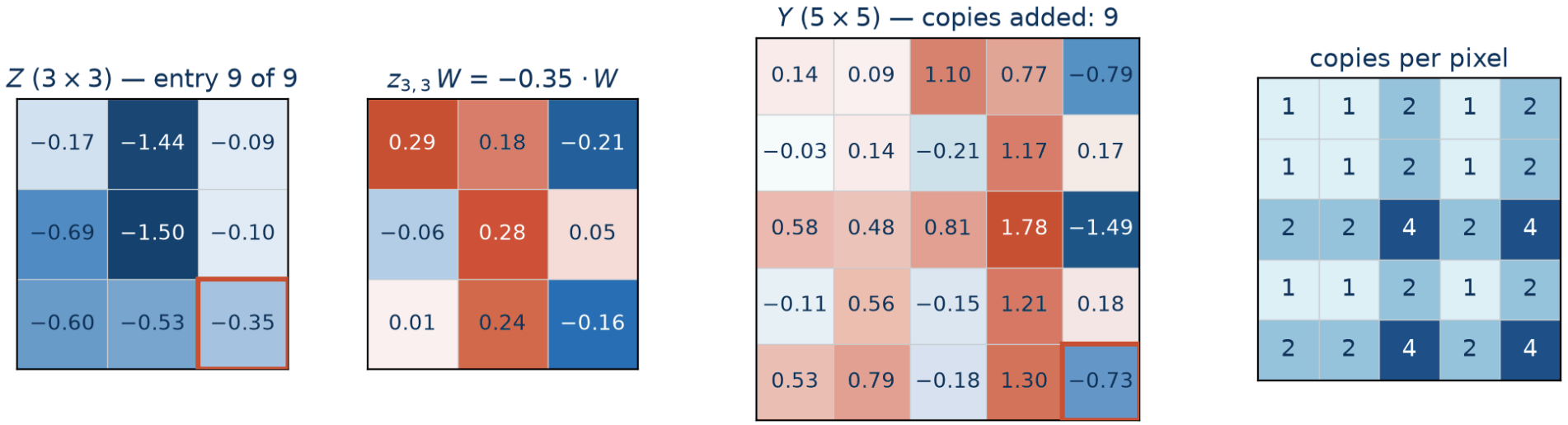


X (5 × 5) → Z (3 × 3) → Y (5 × 5): the shape is restored, the values are not

The same kernel on a 5×5 map with stride 2: C is $\lceil 5/2 \rceil^2 \times 25 = 9 \times 25$, Z is 3×3 and $C^\top \operatorname{vec}(Z)$ is again 5×5. **The shape is restored, the values are not** — the relative error against X is $122\,\%$.

code example — the convolution matrix, checked against `scipy.signal.correlate2d`

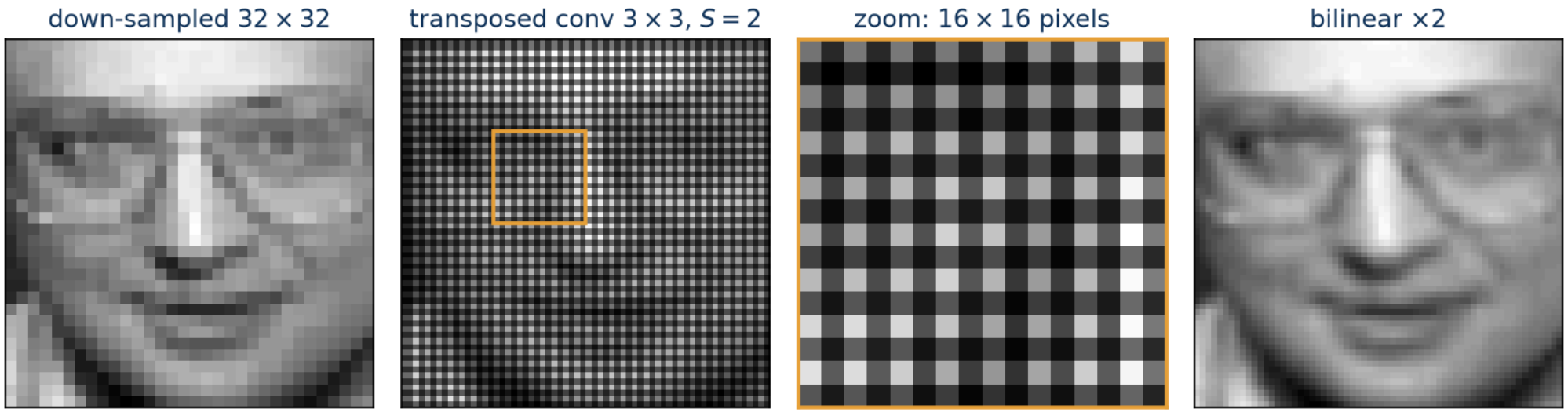
Transposed convolution — what the product does



stride 2 puts a 3 × 3 kernel copy at every second pixel: the copies overlap unevenly (1, 2 or 4 per pixel)

$C^\top \operatorname{vec}(Z)$ is a **scatter**: each entry of Z adds a weighted kernel copy; with 3×3 and stride 2 the copies overlap $1 / 2 / 4$ times → the **checkerboard artefact**.

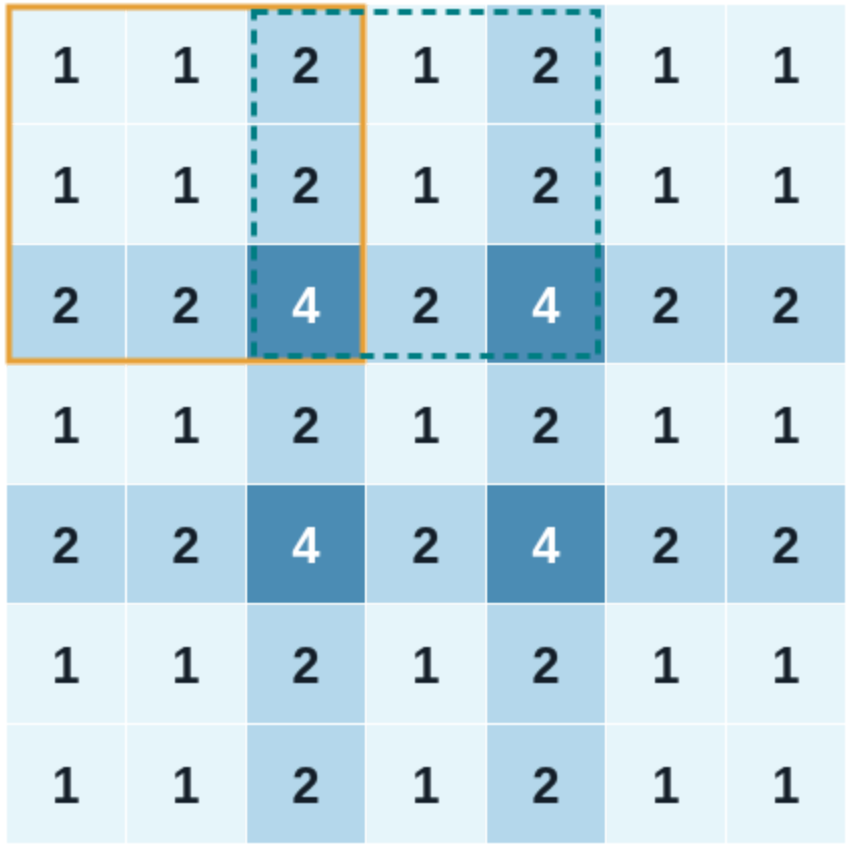
Transposed convolution — the checkerboard on a face



all nine kernel weights = 1, no training; each panel is scaled to its own range

No training, all nine weights 1: the output pixels with **4 copies** are about $4\times$ brighter than those with **1** — the face is buried under the grid. Bilinear up-sampling of the same map is smooth.

Playground: where does the checkerboard come from?



A transposed convolution drops one F × F kernel copy per input pixel, S pixels apart. The numbers = **copies per output pixel**. first copy second copy

input map N × N, **N** = 3

kernel **F** × F, F = 3

stride **S** = 2

3 × 3, S = 2 (slide) | 4 × 4, S = 2 | 2 × 2, S = 2

output: (N − 1)·S + F = **7 × 7**

copies → pixels: 1 → 25, 2 → 20, 4 → 4

S does not divide F: uneven overlap → checkerboard

Long skips + concatenation = U-Net

- **Encoder:** pooling or strided convolutions shrink the map — the spatial bottleneck
- **Decoder:** unpooling, transposed convolution or interpolation grow it back
- **Long skips:** every encoder map is **concatenated** to the decoder map of the same size

> The bottleneck tells the decoder **what** is in the image, the long skips tell it **where** — the U-Net of the next lecture.

6 · Attention module

> Not every pixel and not every feature map deserves the same weight.



Illustration generated by AI (FLUX.2)

Attention module

- first the **global context**, then the details at full resolution
- **Attention map:** a learned weight map (values 0–1) that says where the area of interest is; it **multiplies** the activation map
- computed **from the content**, unlike the fixed weights of a convolution



Illustration generated by AI (FLUX.2)

✓ Pros

- context information for little computation
- spatial context for **segmentation**
- the basic unit of **transformers** (self-attention)

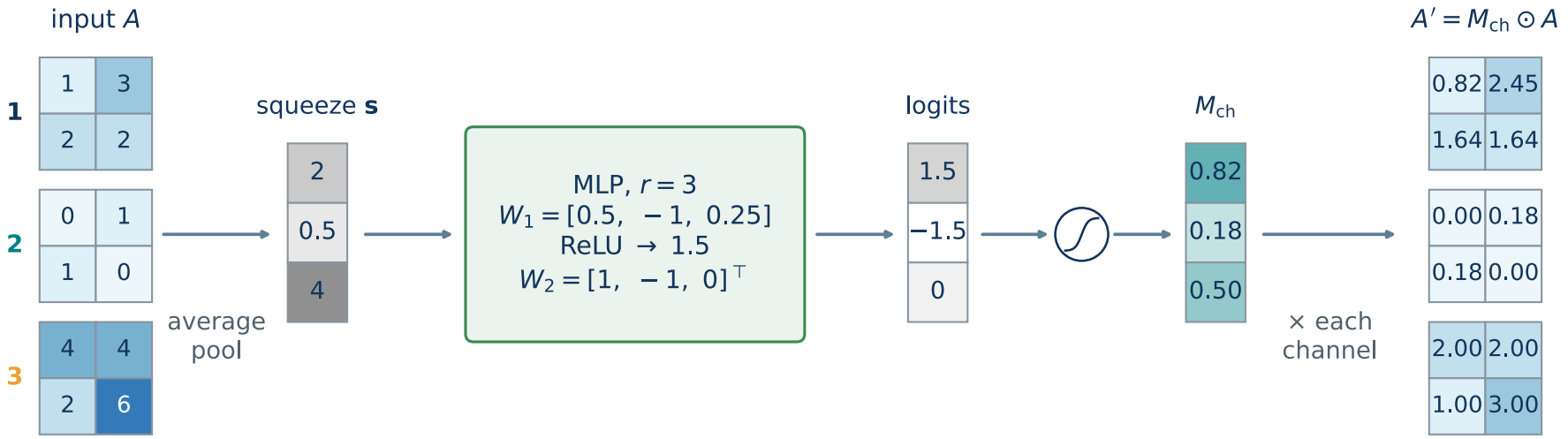
✕ Cons

- self-attention costs grow **quadratically** with the number of positions

Channel attention — squeeze and excitation

Squeeze: global average pooling, one number s_d per channel · **Excitation:** a small multilayer perceptron (MLP) $D \to D/r \to D$ and a sigmoid · **Rescale:** every channel times its weight

$$M_{\text{ch}} = \sigma\big(W_2\,\text{ReLU}(W_1\mathbf{s})\big), \qquad A' = M_{\text{ch}} \odot A$$



Hu et al., *Squeeze-and-Excitation Networks*, CVPR 2018 (arXiv:1709.01507).

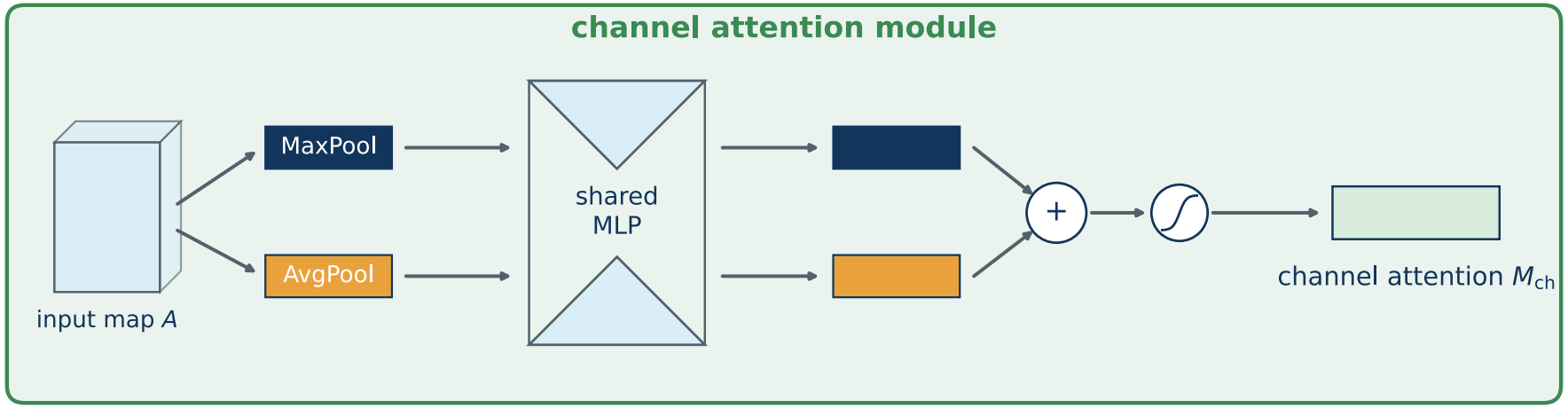
Channel attention in CBAM

Input
map $A, W \times H \times D$

Output
M_{ch}: one weight per channel, $1 \times 1 \times D$

Hyperparameters
MLP size (reduction ratio r)

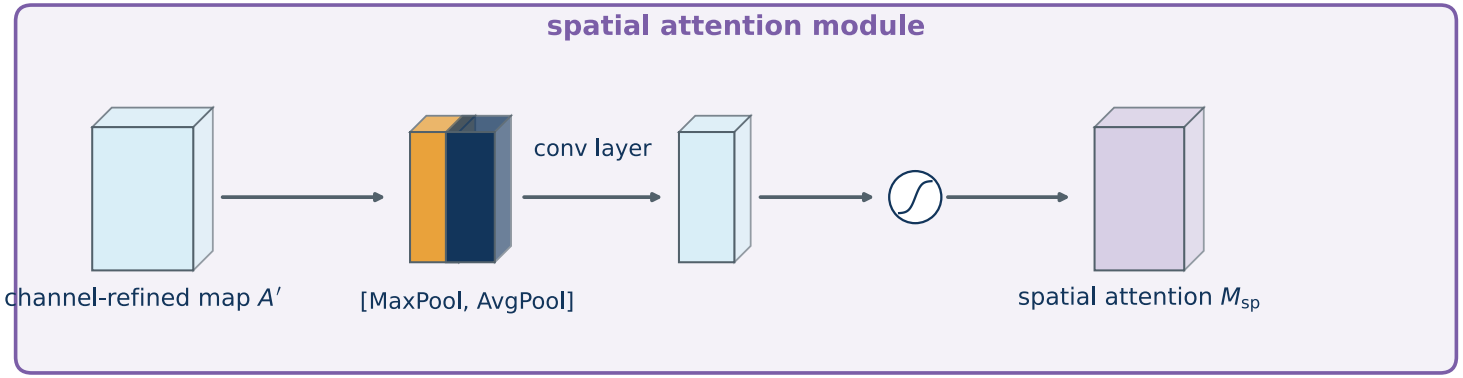
Learned parameters
the shared MLP

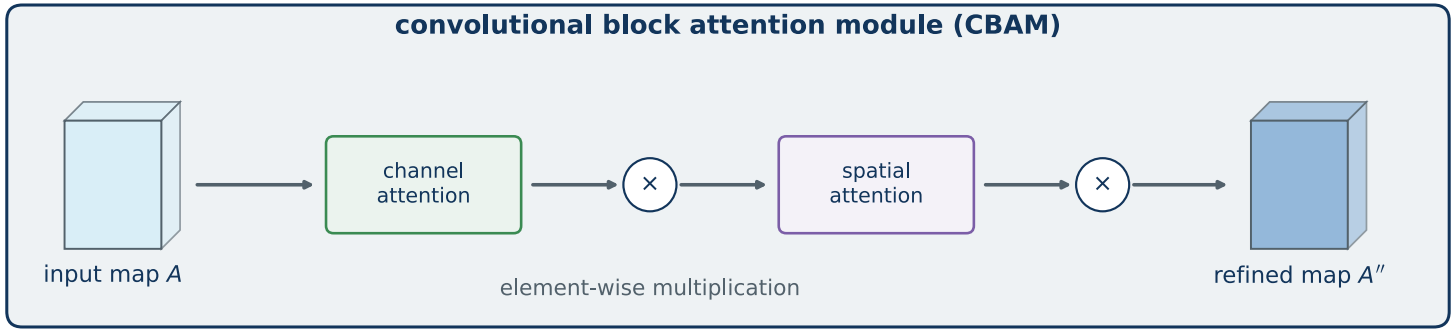


CBAM (convolutional block attention module) squeezes each channel **twice** — global max and average pooling — sends both vectors through the **same** MLP and adds the two scores before the sigmoid.

Drawn after Woo et al., *CBAM: Convolutional Block Attention Module*, ECCV 2018 (arXiv:1807.06521).

Spatial attention and CBAM





Drawn after Woo et al., *CBAM: Convolutional Block Attention Module*, ECCV 2018 (arXiv:1807.06521).

The same blocks in PyTorch


```
# residual block (a): 74 112 parameters
f = nn.Sequential(
    nn.Conv2d(64, 64, 3, padding=1),
    nn.BatchNorm2d(64), nn.ReLU(),
    nn.Conv2d(64, 64, 3, padding=1),
    nn.BatchNorm2d(64))
y = torch.relu(f(x) + x)

nn.Conv2d(256, 64, 1)      # 1 × 1: 16 448
# bottleneck 256 → 64 → 64 → 256: 69 632
nn.Sequential(nn.Conv2d(256, 64, 1, bias=False),
    nn.Conv2d(64, 64, 3, padding=1, bias=False),
    nn.Conv2d(64, 256, 1, bias=False))
```

```
# δ = 2, P = 2: 36 928 parameters, W × H kept
nn.Conv2d(64, 64, 3, padding=2, dilation=2)

pool = nn.MaxPool2d(2, return_indices=True)
z, idx = pool(x)                  # idx = switches
u = nn.MaxUnpool2d(2)(z, idx)     # 0 parameters

# 16 × 16 → 32 × 32: 18 464 parameters
nn.ConvTranspose2d(64, 32, 3, stride=2,
                   padding=1, output_padding=1)

# SE block, r = 16: 580 parameters
se = nn.Sequential(nn.AdaptiveAvgPool2d(1),
    nn.Conv2d(64, 4, 1), nn.ReLU(),
    nn.Conv2d(4, 64, 1), nn.Sigmoid())
y = se(x) * x
```

Tensors are (N, C, H, W): C = our depth D. `kernel_size` = F, `stride` = S, `padding` = P, `dilation` = δ, `out_channels` = K.

7 · Recap

> Today’s blocks are the ingredients — most of the architectures of the next lecture are cooked from them.



Illustration generated by AI (FLUX.2)

Summary — the six blocks

1 · Skip connections

- carry information and gradient past several layers
- joined by concatenation or addition

2 · Residual block

- learns $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$
- the identity is easy, so depth hurts much less

3 · 1 × 1 convolution, bottleneck

- mixes features in depth only
- 512 → 64 → 64 → 512: 23 × cheaper (ResNet-50 block)

4 · Dilated convolution

- larger receptive field, same 9 weights
- no down-sampling, full resolution kept

5 · Shrink and grow back

- autoencoder, spatial bottleneck
- unpooling, transposed convolution, interpolation → U-Net

6 · Attention

- a learned 0–1 map multiplies the activations
- channel (M_{ch}) and spatial (M_{sp})

Recap quiz I — blocks 1 to 3

Skip connections

- What are the two ways of **joining** activation maps? What must hold about the tensor shapes in each, and what happens to the depth?

Residual block

- Write the definition of the **residual** $\mathcal{F}(\mathbf{x})$. Why is it easier to learn than $\mathcal{H}(\mathbf{x})$ itself?
- Why does a residual branch help against **vanishing gradients**? Which term appears in the Jacobian?
- By hand: $y = f(wx)$ against $y = x + f(wx)$ with $f = \tanh, w = 0.1, x = 0$. What is dy/dx for each, and for a chain of ten such layers?

1 × 1 convolution and bottleneck

- A 1×1 convolution turns $W \times H \times 16$ into $W \times H \times 4$. How many weights and biases does it have?
- Why is a 1×1 convolution on a $1 \times 1 \times D$ tensor the same thing as a fully connected layer?
- In the bottleneck block $512 \to 64 \to 64 \to 512$ — where exactly does the saving come from, and what is the price?

Recap quiz II — blocks 4 to 6

Dilated convolution

- What does the dilation δ multiply in the formula — the position (i, j) in the **image** or the **kernel offsets** u, v? What would the other option do?
- A chain of 3×3 convolutions with $\delta = 1, 2, 4$ — what is the receptive field, and how many weights does it use?

Shrink and grow back

- What are **switch variables**? How many of the 16 values of a 4×4 map survive a 2×2 pooling?
- What is the size of the convolution matrix C for $M = 8$, stride 1 and stride 2? Why can it not be inverted in the second case?
- Why is “deconvolution” the wrong name for a transposed convolution?
- By hand: the 1D transposed convolution of $[a, b]$, kernel $[1, 2, 1]$, stride 2?

Attention

- **Channel** versus **spatial** attention: what is pooled away in each, and what is the shape of the resulting attention map?